DESARROLLO DE APLICACIONES CON OBJETOS · TPI · PATRONES DE DISEÑO

Patrones Strategy, State e Iterator

Los ejercicios propuestos de la cátedra resueltos: los 6 del TP y los 3 de la guía, con su salida

Strategy | State | Iterator | 9 ejercicios probados

Qué hay en esta guía

Los ejercicios propuestos de la cátedra para los **patrones de comportamiento**, resueltos y probados: los **6 del TP** (dos por patrón) y los **3 de la guía** (uno por patrón). Cada uno tiene el enunciado tal cual, cómo se piensa, el código completo y la salida real al correrlo. Los ejemplos resueltos (descuentos, pedido, biblioteca, calificaciones, semáforo y tienda) ya están explicados en la guía y la clase de la cátedra.

0 Los tres patrones, en una hoja

Patrón	Pregunta que responde	Cómo se arma	La línea clave
Strategy	¿Qué algoritmo uso?	Una clase abstracta con un método y una clase por algoritmo; el «contexto» guarda una estrategia y la puede cambiar	`self.estrategia.calcular(...)`
State	¿Qué puede hacer el objeto ahora?	Una clase por estado; cada acción del objeto se la pasa al estado actual, y el estado decide si cambia a otro	`self.estado.avanzar(self)` y `cambiar_estado(Otro())`
Iterator	¿Cómo recorro los elementos?	La colección tiene `__iter__()`, que devuelve un iterador con `__next__()`; al terminar, `raise StopIteration`	`for x in coleccion`

Strategy y State se parecen

Los dos delegan en un objeto con una interfaz común. La diferencia es la intención: en **Strategy** el que usa la clase elige el algoritmo desde afuera; en **State** es el propio estado el que decide a qué estado pasar.

TP · Strategy, State e Iterator

1 Cálculo de salarios

Strategy | TP · Ejercicio 1 | Solución propuesta

Enunciado (tal cual)

Crear EstrategiaSalario y las estrategias SalarioFijo, SalarioPorHoras y SalarioConComision. Crear CalculadoraSalario, capaz de cambiar de estrategia sin usar condicionales. Probar: fijo $500.000; 40 horas a $8.000 = $320.000; básico $300.000 más 10% de ventas por $1.000.000 = $400.000.

Cómo se piensa

- Cada forma de pagar es una clase con el mismo método, `calcular()`, y guarda sus propios datos (monto, horas, ventas).
- `CalculadoraSalario` solo tiene la estrategia y le pide `calcular()`: no hay ningún `if` que pregunte qué tipo de salario es.
- Cambiar de estrategia es asignar otro objeto con `cambiar_estrategia()`.

tp1_salarios.py

```
"""TP · Ejercicio 1 — Cálculo de salarios (Strategy)."""
from abc import ABC, abstractmethod


class EstrategiaSalario(ABC):
    @abstractmethod
    def calcular(self):
        pass


class SalarioFijo(EstrategiaSalario):
    def __init__(self, monto):
        self.monto = monto

    def calcular(self):
        return self.monto


class SalarioPorHoras(EstrategiaSalario):
    def __init__(self, horas, valor_hora):
        self.horas = horas
        self.valor_hora = valor_hora

    def calcular(self):
        return self.horas * self.valor_hora


class SalarioConComision(EstrategiaSalario):
    def __init__(self, basico, ventas, porcentaje):
        self.basico = basico
        self.ventas = ventas
        self.porcentaje = porcentaje
```

```
    def calcular(self):
        return self.basico + self.ventas * self.porcentaje / 100


class CalculadoraSalario:
    def __init__(self, estrategia):
        self.estrategia = estrategia

    def cambiar_estrategia(self, estrategia):
        self.estrategia = estrategia

    def calcular(self):
        # Polimorfismo: no pregunta qué estrategia es, le pide que calcule
        return self.estrategia.calcular()


def pesos(x):
    # Formato argentino: punto para los miles
    return "$" + f"{x:,.0f}".replace(",", ".")


if __name__ == "__main__":
    calculadora = CalculadoraSalario(SalarioFijo(500_000))
    print("Fijo:         ", pesos(calculadora.calcular()))
    calculadora.cambiar_estrategia(SalarioPorHoras(40, 8_000))
    print("Por horas:    ", pesos(calculadora.calcular()))
    calculadora.cambiar_estrategia(SalarioConComision(300_000, 1_000_000, 10))
    print("Con comisión: ", pesos(calculadora.calcular()))
```

Salida al correrlo

```
Fijo:          $500.000
Por horas:     $320.000
Con comisión:  $400.000
```

2 Sistema de pagos

Strategy · TP · Ejercicio 2 · Solución propuesta

Enunciado (tal cual)

Implementar PagoEfectivo (10% de descuento), PagoTransferencia (sin cambios) y PagoTarjeta (15% de recargo), todas con una interfaz común. Crear ProcesadorPago. Agregar luego PagoPromocional (20% de descuento) sin modificar ProcesadorPago.

Cómo se piensa

- La interfaz común es la clase abstracta `MedioPago` con `total(monto)`.
- `PagoPromocional` se agrega al final como una clase más: `ProcesadorPago` no se toca porque solo conoce la interfaz. Es la gracia del patrón (abierto a extensión, cerrado a modificación).

tp2_pagos.py

```
"""TP · Ejercicio 2 — Sistema de pagos (Strategy)."""
from abc import ABC, abstractmethod
```

```python
class MedioPago(ABC):
    @abstractmethod
    def total(self, monto):
        pass


class PagoEfectivo(MedioPago):
    def total(self, monto):
        return monto * 0.90          # 10 % de descuento


class PagoTransferencia(MedioPago):
    def total(self, monto):
        return monto                 # sin cambios


class PagoTarjeta(MedioPago):
    def total(self, monto):
        return monto * 1.15          # 15 % de recargo


class ProcesadorPago:
    def __init__(self, medio):
        self.medio = medio

    def cambiar_medio(self, medio):
        self.medio = medio

    def procesar(self, monto):
        return self.medio.total(monto)


# Se agrega después, sin tocar ProcesadorPago: alcanza con una clase nueva con la misma interfaz
class PagoPromocional(MedioPago):
    def total(self, monto):
        return monto * 0.80          # 20 % de descuento


def pesos(x):
    # Formato argentino: punto para los miles
    return "$" + f"{x:,.0f}".replace(",", ".")


if __name__ == "__main__":
    monto = 10_000
    procesador = ProcesadorPago(PagoEfectivo())
    for medio in [PagoEfectivo(), PagoTransferencia(), PagoTarjeta(), PagoPromocional()]:
        procesador.cambiar_medio(medio)
        print(f"{type(medio).__name__:<18} {pesos(procesador.procesar(monto))}")
```

Salida al correrlo

```
PagoEfectivo       $9.000
PagoTransferencia  $10.000
PagoTarjeta        $11.500
PagoPromocional    $8.000
```

3 Reproductor de música

State | TP · Ejercicio 3 | Solución propuesta

Enunciado (tal cual)

Implementar los estados Detenido, Reproduciendo y Pausado, con operaciones reproducir(), pausar() y detener(). Detenido + reproducir → Reproduciendo; Reproduciendo + pausar → Pausado; Pausado + reproducir → Reproduciendo; Reproduciendo o Pausado + detener → Detenido. Las acciones no permitidas deben informar el problema sin cambiar el estado.

Cómo se piensa

- La clase base del estado trae las tres acciones con el comportamiento «no permitido»: avisa y no cambia nada.
- Cada estado redefine solo las acciones que sí puede hacer y ahí llama a `reproductor.cambiar_estado(...)`.
- `Reproductor` no tiene ningún `if`: le pasa cada acción al estado actual con `self.estado.accion(self)`.

tp3_reproductor.py

```
"""TP · Ejercicio 3 — Reproductor de música (State)."""
from abc import ABC, abstractmethod


class EstadoReproductor(ABC):
    # Por defecto, cada acción no está permitida: avisa y el estado no cambia
    def reproducir(self, reproductor):
        print(f"  No se puede reproducir: está {self}")

    def pausar(self, reproductor):
        print(f"  No se puede pausar: está {self}")

    def detener(self, reproductor):
        print(f"  No se puede detener: está {self}")

    @abstractmethod
    def __str__(self):
        pass


class Detenido(EstadoReproductor):
    def reproducir(self, reproductor):
        print("  Reproduciendo")
        reproductor.cambiar_estado(Reproduciendo())

    def __str__(self):
        return "detenido"


class Reproduciendo(EstadoReproductor):
    def pausar(self, reproductor):
        print("  En pausa")
        reproductor.cambiar_estado(Pausado())
```

```
    def detener(self, reproductor):
        print("  Detenido")
        reproductor.cambiar_estado(Detenido())

    def __str__(self):
        return "reproduciendo"


class Pausado(EstadoReproductor):
    def reproducir(self, reproductor):
        print("  Sigue la reproducción")
        reproductor.cambiar_estado(Reproduciendo())

    def detener(self, reproductor):
        print("  Detenido")
        reproductor.cambiar_estado(Detenido())

    def __str__(self):
        return "en pausa"


class Reproductor:
    def __init__(self):
        self.estado = Detenido()

    def cambiar_estado(self, nuevo_estado):
        self.estado = nuevo_estado

    def reproducir(self):
        self.estado.reproducir(self)

    def pausar(self):
        self.estado.pausar(self)

    def detener(self):
        self.estado.detener(self)


if __name__ == "__main__":
    r = Reproductor()
    for accion in ["pausar", "reproducir", "reproducir", "pausar", "pausar", "reproducir",
"detener", "detener"]:
        print(f"{accion}() estando {r.estado}:")
        getattr(r, accion)()
    print("Estado final:", r.estado)
```

Salida al correrlo

```
pausar() estando detenido:
  No se puede pausar: está detenido
reproducir() estando detenido:
  Reproduciendo
reproducir() estando reproduciendo:
  No se puede reproducir: está reproduciendo
pausar() estando reproduciendo:
  En pausa
pausar() estando en pausa:
  No se puede pausar: está en pausa
reproducir() estando en pausa:
  Sigue la reproducción
detener() estando reproduciendo:
  Detenido
detener() estando detenido:
  No se puede detener: está detenido
Estado final: detenido
```

4 Préstamo de biblioteca

State | TP · Ejercicio 4 | Solución propuesta

Enunciado (tal cual)

Crear Libro con estados Disponible, Prestado y EnReparacion. Implementar prestar(), devolver(), enviar_reparacion() y finalizar_reparacion(). Un libro prestado debe devolverse antes de poder prestarlo otra vez; uno en reparación no puede prestarse. Cada estado decide qué acciones permite.

Cómo se piensa

- Mismo armado que el reproductor: la base avisa «no se puede» y cada estado redefine lo suyo.
- Disponible permite prestar y enviar a reparación; Prestado, solo devolver; EnReparacion, solo finalizar la reparación.
- Un libro prestado tampoco se manda a reparación: primero hay que devolverlo.

tp4_biblioteca.py

```python
"""TP · Ejercicio 4 — Préstamo de biblioteca (State)."""
from abc import ABC, abstractmethod


class EstadoLibro(ABC):
    # Cada estado redefine solo lo que permite; lo demás avisa sin cambiar el estado
    def prestar(self, libro):
        print(f"  No se puede prestar: está {self}")

    def devolver(self, libro):
        print(f"  No se puede devolver: está {self}")

    def enviar_reparacion(self, libro):
        print(f"  No se puede enviar a reparación: está {self}")

    def finalizar_reparacion(self, libro):
```

```
        print(f"  No se puede finalizar la reparación: está {self}")

    @abstractmethod
    def __str__(self):
        pass


class Disponible(EstadoLibro):
    def prestar(self, libro):
        print("  Prestado")
        libro.cambiar_estado(Prestado())

    def enviar_reparacion(self, libro):
        print("  Enviado a reparación")
        libro.cambiar_estado(EnReparacion())

    def __str__(self):
        return "disponible"


class Prestado(EstadoLibro):
    def devolver(self, libro):
        print("  Devuelto")
        libro.cambiar_estado(Disponible())

    def __str__(self):
        return "prestado"


class EnReparacion(EstadoLibro):
    def finalizar_reparacion(self, libro):
        print("  Reparación terminada")
        libro.cambiar_estado(Disponible())

    def __str__(self):
        return "en reparación"


class Libro:
    def __init__(self, titulo):
        self.titulo = titulo
        self.estado = Disponible()

    def cambiar_estado(self, nuevo_estado):
        self.estado = nuevo_estado

    def prestar(self):
        self.estado.prestar(self)

    def devolver(self):
        self.estado.devolver(self)

    def enviar_reparacion(self):
        self.estado.enviar_reparacion(self)

    def finalizar_reparacion(self):
        self.estado.finalizar_reparacion(self)


if __name__ == "__main__":
    libro = Libro("Rayuela")
```

```
    for accion in ["prestar", "prestar", "enviar_reparacion", "devolver", "enviar_reparacion",
                   "prestar", "finalizar_reparacion", "prestar"]:
        print(f"{accion}() estando {libro.estado}:")
        getattr(libro, accion)()
    print(f"«{libro.titulo}» queda {libro.estado}")
```

Salida al correrlo

```
prestar() estando disponible:
  Prestado
prestar() estando prestado:
  No se puede prestar: está prestado
enviar_reparacion() estando prestado:
  No se puede enviar a reparación: está prestado
devolver() estando prestado:
  Devuelto
enviar_reparacion() estando disponible:
  Enviado a reparación
prestar() estando en reparación:
  No se puede prestar: está en reparación
finalizar_reparacion() estando en reparación:
  Reparación terminada
prestar() estando disponible:
  Prestado
«Rayuela» queda prestado
```

5 Alumnos aprobados

Iterator | TP · Ejercicio 5 | Solución propuesta

Enunciado (tal cual)

Crear Alumno (nombre, nota), Curso e IteradorAprobados. El iterador debe entregar únicamente alumnos con nota ≥ 6, sin filtrar en el programa principal. Probar: Ana 8, Luis 4, María 7, Pedro 3 y Sofía 9. Deben mostrarse Ana, María y Sofía.

Cómo se piensa

- `Curso.__iter__()` devuelve un `IteradorAprobados`: por eso `for alumno in curso` ya sale filtrado.
- En `__next__()` un `while` saltea a los que tienen menos de 6; cuando no quedan, `raise StopIteration`.

tp5_aprobados.py

```
"""TP · Ejercicio 5 — Alumnos aprobados (Iterator)."""


class Alumno:
    def __init__(self, nombre, nota):
        self.nombre = nombre
        self.nota = nota

    def __str__(self):
        return f"{self.nombre} ({self.nota})"
```

```python
class IteradorAprobados:
    def __init__(self, alumnos):
        self.alumnos = alumnos
        self.posicion = 0

    def __iter__(self):
        return self

    def __next__(self):
        # Salta los que no aprobaron: el filtro está acá, no en el programa principal
        while self.posicion < len(self.alumnos):
            alumno = self.alumnos[self.posicion]
            self.posicion += 1
            if alumno.nota >= 6:
                return alumno
        raise StopIteration


class Curso:
    def __init__(self):
        self.alumnos = []

    def agregar(self, alumno):
        self.alumnos.append(alumno)

    def __iter__(self):
        return IteradorAprobados(self.alumnos)


if __name__ == "__main__":
    curso = Curso()
    for nombre, nota in [("Ana", 8), ("Luis", 4), ("María", 7), ("Pedro", 3), ("Sofía", 9)]:
        curso.agregar(Alumno(nombre, nota))
    print("Aprobados:")
    for alumno in curso:
        print(" ", alumno)
```

Salida al correrlo

```
Aprobados:
  Ana (8)
  María (7)
  Sofía (9)
```

6 Recorrido inverso de tareas

Iterator | TP · Ejercicio 6 | Solución propuesta

Enunciado (tal cual)

Crear Tarea, ListaTareas e IteradorInverso. Agregar las tareas “Estudiar Python”, “Preparar parcial” y “Entregar trabajo”. Recorrerlas desde la última hasta la primera mediante un for, sin acceder directamente a la lista interna desde el programa principal.

Cómo se piensa

- El iterador arranca en la última posición (`len(tareas) - 1`) y va restando.

- La lista interna es `_tareas` (con guion bajo: «no se toca desde afuera»); el programa principal solo hace `for tarea in lista`.

tp6_tareas_inverso.py

```
"""TP · Ejercicio 6 — Recorrido inverso de tareas (Iterator)."""


class Tarea:
    def __init__(self, descripcion):
        self.descripcion = descripcion

    def __str__(self):
        return self.descripcion


class IteradorInverso:
    def __init__(self, tareas):
        self.tareas = tareas
        self.posicion = len(tareas) - 1      # arranca por la última

    def __iter__(self):
        return self

    def __next__(self):
        if self.posicion < 0:
            raise StopIteration
        tarea = self.tareas[self.posicion]
        self.posicion -= 1
        return tarea


class ListaTareas:
    def __init__(self):
        self._tareas = []

    def agregar(self, tarea):
        self._tareas.append(tarea)

    def __iter__(self):
        return IteradorInverso(self._tareas)


if __name__ == "__main__":
    lista = ListaTareas()
    for texto in ["Estudiar Python", "Preparar parcial", "Entregar trabajo"]:
        lista.agregar(Tarea(texto))
    for n, tarea in enumerate(lista, 1):
        print(f"{n}. {tarea}")
```

Salida al correrlo

```
1. Entregar trabajo
2. Preparar parcial
3. Estudiar Python
```

Guía de patrones de diseño

7 Costos de envío

Strategy | Guía · Ejercicio 1 | Solución propuesta

Enunciado (tal cual)

Crear EstrategiaEnvio (abstracta, con calcular_costo(peso)), EnvioNormal ($1.000/kg), EnvioExpress ($1.500/kg + $2.000 fijos) y EnvioEconomico ($800/kg). Crear CalculadoraEnvios que permita cambiar la estrategia y calcular el costo de un paquete de 5 kg con las tres modalidades.

Cómo se piensa

- Igual al ejemplo de los descuentos: una clase por modalidad con `calcular_costo(peso)`.
- Con 5 kg: normal 5 × 1.000 = $5.000; express 5 × 1.500 + 2.000 = $9.500; económico 5 × 800 = $4.000.

guia1_envios.py

```python
"""Guía · Ejercicio 1 — Costos de envío (Strategy)."""
from abc import ABC, abstractmethod


class EstrategiaEnvio(ABC):
    @abstractmethod
    def calcular_costo(self, peso):
        pass


class EnvioNormal(EstrategiaEnvio):
    def calcular_costo(self, peso):
        return peso * 1_000


class EnvioExpress(EstrategiaEnvio):
    def calcular_costo(self, peso):
        return peso * 1_500 + 2_000


class EnvioEconomico(EstrategiaEnvio):
    def calcular_costo(self, peso):
        return peso * 800


class CalculadoraEnvios:
    def __init__(self, estrategia):
        self.estrategia = estrategia

    def cambiar_estrategia(self, estrategia):
        self.estrategia = estrategia

    def costo(self, peso):
        return self.estrategia.calcular_costo(peso)
```

```python
def pesos(x):
    # Formato argentino: punto para los miles
    return "$" + f"{x:,.0f}".replace(",", ".")


if __name__ == "__main__":
    calculadora = CalculadoraEnvios(EnvioNormal())
    for estrategia in [EnvioNormal(), EnvioExpress(), EnvioEconomico()]:
        calculadora.cambiar_estrategia(estrategia)
        print(f"{type(estrategia).__name__:<15} paquete de 5 kg: {pesos(calculadora.costo(5))}")
```

Salida al correrlo

```
EnvioNormal     paquete de 5 kg: $5.000
EnvioExpress    paquete de 5 kg: $9.500
EnvioEconomico  paquete de 5 kg: $4.000
```

8 Máquina expendedora

State · Guía · Ejercicio 2 · Solución propuesta

Enunciado (tal cual)

Implementar MaquinaExpendedora con los estados SinDinero, ConDinero y SinStock. Operaciones: ingresar_dinero(), comprar() y reponer_stock(). Cada estado debe decidir qué operaciones permite y qué transición produce. Definir explícitamente qué ocurre cuando no hay stock o se intenta comprar sin dinero.

Cómo se piensa

- Lo que decidí (el enunciado pide definirlo): sin dinero no se compra; con dinero, comprar entrega y descuenta el stock; sin stock no se acepta dinero ni se compra.
- Reponer se puede siempre (está en la clase base); `SinStock` lo redefine para, además, volver a `SinDinero`.
- Al comprar el último producto, `ConDinero` pasa a `SinStock` en lugar de `SinDinero`.

guia2_expendedora.py

```python
"""Guía · Ejercicio 2 — Máquina expendedora (State)."""
from abc import ABC, abstractmethod


class EstadoMaquina(ABC):
    @abstractmethod
    def ingresar_dinero(self, maquina):
        pass

    @abstractmethod
    def comprar(self, maquina):
        pass

    def reponer_stock(self, maquina, cantidad):
        # Reponer se puede en cualquier estado; si estaba sin stock, pasa a esperar dinero
        maquina.stock += cantidad
        print(f"  Repuesto: ahora hay {maquina.stock}")
```

```python
class SinDinero(EstadoMaquina):
    def ingresar_dinero(self, maquina):
        print("  Dinero ingresado")
        maquina.cambiar_estado(ConDinero())

    def comprar(self, maquina):
        print("  Primero hay que ingresar dinero")

    def __str__(self):
        return "sin dinero"


class ConDinero(EstadoMaquina):
    def ingresar_dinero(self, maquina):
        print("  Ya hay dinero ingresado: comprá o esperá")

    def comprar(self, maquina):
        maquina.stock -= 1
        print(f"  Producto entregado (quedan {maquina.stock})")
        maquina.cambiar_estado(SinStock() if maquina.stock == 0 else SinDinero())

    def __str__(self):
        return "con dinero"


class SinStock(EstadoMaquina):
    def ingresar_dinero(self, maquina):
        print("  No hay stock: no se acepta dinero")

    def comprar(self, maquina):
        print("  No hay stock")

    def reponer_stock(self, maquina, cantidad):
        super().reponer_stock(maquina, cantidad)
        maquina.cambiar_estado(SinDinero())

    def __str__(self):
        return "sin stock"


class MaquinaExpendedora:
    def __init__(self, stock):
        self.stock = stock
        self.estado = SinDinero() if stock > 0 else SinStock()

    def cambiar_estado(self, nuevo_estado):
        self.estado = nuevo_estado

    def ingresar_dinero(self):
        self.estado.ingresar_dinero(self)

    def comprar(self):
        self.estado.comprar(self)

    def reponer_stock(self, cantidad):
        self.estado.reponer_stock(self, cantidad)


if __name__ == "__main__":
```

```
    m = MaquinaExpendedora(stock=1)
    pasos = [("comprar", ()), ("ingresar_dinero", ()), ("ingresar_dinero", ()), ("comprar", ()),
             ("ingresar_dinero", ()), ("comprar", ()), ("reponer_stock", (2,)),
("ingresar_dinero", ()), ("comprar", ())]
    for accion, args in pasos:
        print(f"{accion}{args if args else '()'} estando {m.estado}:")
        getattr(m, accion)(*args)
    print("Estado final:", m.estado, "· stock:", m.stock)
```

Salida al correrlo

```
comprar() estando sin dinero:
  Primero hay que ingresar dinero
ingresar_dinero() estando sin dinero:
  Dinero ingresado
ingresar_dinero() estando con dinero:
  Ya hay dinero ingresado: comprá o esperá
comprar() estando con dinero:
  Producto entregado (quedan 0)
ingresar_dinero() estando sin stock:
  No hay stock: no se acepta dinero
comprar() estando sin stock:
  No hay stock
reponer_stock(2,) estando sin stock:
  Repuesto: ahora hay 2
ingresar_dinero() estando sin dinero:
  Dinero ingresado
comprar() estando con dinero:
  Producto entregado (quedan 1)
Estado final: sin dinero · stock: 1
```

9 Curso y alumnos

Iterator | Guía · Ejercicio 3 | Solución propuesta

Enunciado (tal cual)

Crear Alumno(nombre, nota), Curso (almacena alumnos) e IteradorAlumnos. Permitir recorrer con for alumno in curso. Incorporar un segundo iterador que entregue solamente los alumnos con nota mayor o igual a 6. El programa principal no debe recorrer directamente la lista interna del curso.

Cómo se piensa

- `for alumno in curso` usa `IteradorAlumnos` (todos); `curso.aprobados()` devuelve el segundo iterador.
- `IteradorAprobados` hereda de `IteradorAlumnos` y reusa su `__next__()` con `super()`, salteando a los que no llegan a 6.

guia3_curso.py

```
"""Guía · Ejercicio 3 — Curso y alumnos (Iterator), con un segundo iterador de aprobados."""


class Alumno:
    def __init__(self, nombre, nota):
        self.nombre = nombre
```

```
        self.nota = nota

    def __str__(self):
        return f"{self.nombre} ({self.nota})"


class IteradorAlumnos:
    def __init__(self, alumnos):
        self.alumnos = alumnos
        self.posicion = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.posicion >= len(self.alumnos):
            raise StopIteration
        alumno = self.alumnos[self.posicion]
        self.posicion += 1
        return alumno


class IteradorAprobados(IteradorAlumnos):
    # Reusa el recorrido y saltea los que tienen menos de 6
    def __next__(self):
        while True:
            alumno = super().__next__()      # al terminar, StopIteration sale de acá
            if alumno.nota >= 6:
                return alumno


class Curso:
    def __init__(self):
        self._alumnos = []

    def agregar(self, alumno):
        self._alumnos.append(alumno)

    def __iter__(self):
        return IteradorAlumnos(self._alumnos)

    def aprobados(self):
        return IteradorAprobados(self._alumnos)


if __name__ == "__main__":
    curso = Curso()
    for nombre, nota in [("Ana", 8), ("Luis", 4), ("María", 7), ("Pedro", 3), ("Sofía", 9)]:
        curso.agregar(Alumno(nombre, nota))
    print("Todos:", ", ".join(str(a) for a in curso))
    print("Aprobados:", ", ".join(str(a) for a in curso.aprobados()))
```

Salida al correrlo

```
Todos: Ana (8), Luis (4), María (7), Pedro (3), Sofía (9)
Aprobados: Ana (8), María (7), Sofía (9)
```